

Optimising Compilers

COMS10015

Dr Tom Deakin

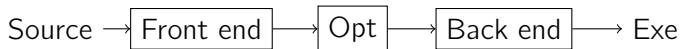
University of Bristol

Week 23

Compiler Optimisations

Compiler

- ▶ Compiler front end converts program source into an intermediate representation (IR).
- ▶ Compiler can then optimise the IR.
- ▶ Needs to make sure the transformations are semantically equivalent.
- ▶ That is, the program still computes the same thing.



Things to optimise for

- ▶ Usually optimise to minimise execution time.
- ▶ Could optimise to minimise binary size,
- ▶ or optimise to minimise power consumption on embedded processors.
- ▶ Each optimisation pass usually includes a cost model to determine if transformation is beneficial for the goal.

Compiler optimisation flags

- ▶ Compilers support many different optimisations.
- ▶ Can usually turn each of them on using a specific compiler flag.
- ▶ To help make good choices, compilers usually offer combine them into groups.
- ▶ E.g. optimisation levels `-O1`, `-O2`, `-O3`, `-Ofast`
- ▶ Read the documentation of the compiler to find out what they might do.
- ▶ Useful to check the default — not all compilers optimise by default!

Compiler optimisation of code

- ▶ Optimising for performance helps reduce the execution time of the programs.
- ▶ Can see big performance improvements without changing any program code!
- ▶ E.g. For a simple explicit heat equation MMS code, built with GCC 9.1 and run on a laptop:

Flag	Runtime (s)	Speedup
-O0	8.32	1.0X
-O1	2.96	2.8X
-O2	2.94	2.8X
-O3	1.82	4.6X
-Ofast	1.51	5.5X
-Ofast -mtune=native	1.37	6.1X

IR transformations

- ▶ During the optimisation phase, the compiler can make multiple passes over the IR.
- ▶ Typically transforms it in place, replacing parts with optimised parts.
- ▶ IR often groups functional parts of the program in *Basic Blocks*, and relationship between them (branches/jumps).
- ▶ Basic blocks don't contain any branches.
- ▶ Optimisations move things around and change things within and/or between blocks.
- ▶ The IR provides the flexibility to do this.

Common subexpression elimination

- ▶ Find identical expressions, and only evaluate them once.
- ▶ More optimal if storing the value is cheaper than computing it multiple times.

Before

```
a = 2 * b + c + d;  
e = 2 * b + f + g;
```

After

```
tmp = 2 * b;  
a = tmp + c + d;  
e = tmp + f + g;
```

Dead code elimination

- ▶ Sometimes code contains unused variables.
- ▶ Or statements where it is impossible for them to be reached.
- ▶ The compiler can safely remove them from the program.

Before

```
int a = 4;  
int b = 2;  
return 2 * a;  
a = a * 3;
```

After

```
int a = 4;  
return 2 * a;
```

Constant folding

- ▶ Programs often contain literals and constant values.
- ▶ The compiler could evaluate the expression into a new literal/constant value.
- ▶ Computation at compile time instead of at program execution time.

Before

```
const double pi = 3.14;  
rad = 2.0 * pi * r * r;
```

After

```
rad = 6.28 * r * r;
```

Hoist loop invariants

- ▶ A loop invariant is a property which is true before the loop, and true after every iteration.
- ▶ Expressions which don't change value are loop invariant, so can be safely moved (hoisted) outside the loop.
- ▶ Saves repeated computation of the same expression every iterations.

Before

```
for (int i = 0; i < N; ++i)
{
    a[i] = x + y;
}
```

After

```
float tmp = x + y;
for (int i = 0; i < N; ++i)
{
    a[i] = tmp;
}
```

Function inlining

- ▶ Function calls require pushing arguments to the stack and a jump instruction to the entry point.
- ▶ Overheads usually small, but large jumps can cause issues for branch predictor, instruction caches, ...
- ▶ Compiler removes the subroutine call by *inlining* the function.
- ▶ Results in a larger code size if function inlined many times.
- ▶ Compilers typically do this if the function is small.

Before

```
float square(float a) {  
    return a * a;  
}  
float c = square(a) + square(b);
```

After

```
float c = a * a + b * b;
```

Strength reduction

- ▶ Replace expensive operations with cheaper operations.
- ▶ Examples:
 - ▶ $x^2 = x * x$, where multiplication is faster than exponential instruction.
 - ▶ $x * 3 = x + x + x$, where addition is faster than multiplication¹.
 - ▶ $x * 2 = x \ll 1$, where shifts are faster than multiplication.
- ▶ Extend idea to optimise loops.

¹On Skylake, MUL has a latency of 3 cycles and ADD has a latency of 1 cycle.

Loop variables

Definitions

Loop invariant A value that is constant across all loop iterations.

Loop iteration variable Variable that tracks iteration number (e.g., i in `for (i = 0; i < N; ++i)`)

Loop induction variable A variable that is a linear function of the iteration variable. (e.g., $a \times i + b$).

For example:

```
for (int i = 0; i < N; ++i)
    A[i] = i * 3; // i*3 is a loop induction variable
```

Strength reduction procedure

A loop has an induction variable i which is updated by $i = i + c$ every iteration, for loop invariant variable c .

A loop induction variable $j = a \times i + b$, a linear function on i in the loop where a, b are loop invariant, can be optimised as follows:

1. Make new induction variable k .
2. Replace use of old induction variable j with new variable k .
3. At end of loop, update $k = k + a \times c$.
4. Initialise the new induction variable k before the loop to $a \times i + b$ for initial value of i .

The value of $a \times c$ is loop invariant, so the multiplication in step 3 can be hoisted above loop.

Strength reduction example

Example from <https://www.cs.cornell.edu/courses/cs6120/2019fa/blog/strength-reduction-pass-in-llvm/>

```
int i = 0;
while( i < 10 ) {
    int j = 3 * i + 2;
    a[j] = a[j] - 2;
    i = i + 2;
}
```

Strength reduction example

Example from <https://www.cs.cornell.edu/courses/cs6120/2019fa/blog/strength-reduction-pass-in-llvm/>

```
int i = 0;
while( i < 10 ) {
    int j = 3 * i + 2;
    a[j] = a[j] - 2;
    i = i + 2;
}
```

$i = 0, 2, 4, 6, \dots$

Strength reduction example

Example from <https://www.cs.cornell.edu/courses/cs6120/2019fa/blog/strength-reduction-pass-in-llvm/>

```
int i = 0;
while( i < 10 ) {
    int j = 3 * i + 2;
    a[j] = a[j] - 2;
    i = i + 2;
}
```

$i = 0, 2, 4, 6, \dots$

$j = 2, 8, 14, 20, 26, \dots$

Strength reduction example

Example from <https://www.cs.cornell.edu/courses/cs6120/2019fa/blog/strength-reduction-pass-in-llvm/>

```
int i = 0;
while( i < 10 ) {
    int j = 3 * i + 2;
    a[j] = a[j] - 2;
    i = i + 2;
}
```

$i = 0, 2, 4, 6, \dots$

$j = 2, 8, 14, 20, 26, \dots$

```
int i = 0;
int j = 2; // j = 3 * 0 + 2;
while( i < 10 ) {
    a[j] = a[j] - 2;
    i = i + 2;
    j = j + 6; // j = j + 3 * 2
}
```

Further Reading

- ▶ https://en.wikipedia.org/wiki/Optimizing_compiler
- ▶ <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- ▶ https://en.wikipedia.org/wiki/Common_subexpression_elimination
- ▶ <https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/14/Slides14.pdf>
- ▶ <https://compileroptimizations.com>
- ▶ <https://www.cs.utexas.edu/~pingali/CS380C/2020/lectures/strengthReduction.pdf>
- ▶ <https://youtu.be/vZk7Br6Vh1U> AoCO 9/25 More Loops: Induction Variables